Code559

Front-End Developer Course

MODULE 1

HTML Foundations

Write your first pages by hand, link them together, and put them on the web.

Free lesson: 559solutions.com/code559/module-01-html-foundations/

In this packet

1. **The lesson**
2. **Project: Home and About pages**
3. **Finishing work in Git**
4. **Troubleshooting: five common problems**
5. **Self-assessment**
6. **Answers**

Powered by 559 Solutions

About this module

What this module covers

You learn how the web delivers a page, how to write HTML by hand, how to link pages together, and how to publish your first page on GitHub Pages. This is foundational: every website, framework and page builder ends up as an HTML file a browser reads. You also finish the first week with something live on the web.

Tips for the learner

Some prompts ask you to explain code to yourself, out loud. Others ask you to write something down. Each habit does a different job.

Recite to understand. When a prompt says "out loud," explain the code in your own words, as if you were teaching it. Saying it aloud makes a deeper impression than reading silently. It helps you picture how the pieces of code fit together. And it shows you right away which line you cannot explain yet. That line is the one to review.

Write it down to check yourself. When a prompt says "write down," it is usually a prediction, a choice, or a guess about a bug. A written answer can't change after you see the result, so the comparison is fair and you learn from the difference. Writing also keeps your attention on the lesson and helps you pick out the main points.

Keep one notebook for this course. Use it for your predictions, key terms, steps, mistakes, and questions to come back to. Date each entry and write the module number, so you can find it later. Before you start a new module, read your last few entries and recite one or two of them out loud.

The four badges

Every core concept moves through the same four steps, called IEIA. Each step is marked with its badge.

 - **Introduce** (a lightbulb: a new idea) - You meet the concept: its name, and why it matters to you, in a sentence or two.

 - **Explain** (a speech bubble: the idea in words) - How it works, one idea at a time, in plain words.

 - **Illustrate** (an eye: see it) - A small example you can look at: a code snippet, a real screenshot or a diagram. Its caption, which starts with this badge, tells you what to notice.

 - **Apply** (a keyboard: your turn) - A short task to do right now, before you read on. Often you type or predict something; sometimes you write it down, sketch it, or explain code to yourself out loud.

Rights and credits

© 2026 559 Solutions LLC. This packet is licensed to the purchaser for personal use. Do not share, resell or post it.

Portions adapted from Visual Studio Code documentation (CC BY 3.0 US), GitHub Docs (CC BY 4.0), and GitHub Skills (MIT). Screenshots used with permission from Microsoft.

Contents

About this module 2

Part 1: The lesson 4

- Why this matters 4
- Before you start 4
- Core concepts 5
  - 1. What happens when you visit a web page? 5
  - 2. What are the parts of a URL? 6
  - 3. How do you create, save and see a file in VS Code? 6
  - 4. What does every HTML document need? 7
  - 5. How do headings, paragraphs and lists work? 8
  - 6. How does a link work? 8
  - 7. How does an image work, and what is `alt` for? 9
  - 8. What are semantic elements, and how do pages share navigation? 10
  - 9. How do you publish your first page? 12
- Exercise 14
- Common misconception 18
- Summary 18
- Check your understanding 18
- Before next module 19
- Go further 19
- What to keep 19

Part 2: Project: Home and About pages 20

Part 3: Finishing work in Git 25

Part 4: Troubleshooting: five common problems 27

Part 5: Self-assessment 29

Part 6: Answers 30

About 559 Solutions 33

© 2026 559 Solutions LLC

PART 1

The lesson

Write your first pages by hand, link them together, and put them on the web.

Time: about 2.5–3 hours (estimate)

In this module you will learn:

- How the web works: client, server, request and response
- The parts of a URL: scheme, host, path (and the anchor)
- How to create, save and open files in VS Code, and the save → refresh loop
- The structure of every HTML document: `<!DOCTYPE html>`, `<html lang>`, `<head>`, `<body>`, `<meta charset>`, `<meta name="viewport">`, `<title>`
- Headings, paragraphs and unordered lists
- Links and `href`, including same-page links to an `id`
- Images, `src` and `alt`
- The semantic elements `header`, `nav`, `main`, `section` and `footer`
- Multiple pages, relative paths, and shared navigation you test link by link
- **Publish your first page:** the GitHub Skills *Introduction to GitHub* exercise, then `git init`, your first commit, `gh repo create` and GitHub Pages
- The save routine you use from Module 2 on: `git status`, `git add .`, `git commit`, `git push`, then check the live page

---

Why this matters

Every web page you have ever opened started as a plain text file of HTML. The browser reads that file and turns it into headings, paragraphs, links and pictures. If you can write that file by hand, you understand what every website, framework and page builder is producing under the hood.

This module starts with how a page travels from a server to your screen. Then you write one page and see how two pages link to each other. You end by putting your work on the public web with your own address, so from the first week you have something live you can show someone.

You will not style anything yet. Plain black-and-white pages are the point: structure first, appearance in Module 2.

Before you start

1. Finish Module 0. You need VS Code (AI features off), Edge or Chrome, Git, the GitHub CLI signed in (`gh auth status`), and a `my-site` folder inside your course folder.
2. Make a second folder next to `my-site` called `practice-01`. This is your practice folder for the Exercise. Nothing in it is published directly; in step 11 you copy your finished page into `my-site`.

© 2026 559 Solutions LLC

3. Find one picture you are happy to put on a public website (a photo you took, or a drawing). A photo taken with a phone can also store the place where it was taken, and anyone who downloads it from your site may be able to read that place (Apple Support, *Manage location metadata in Photos*). So do not use a photo taken at your home. Copy your picture into `practice-01` and rename it `image-one.jpg`, all lowercase. If yours is a `.png`, keep the `.png` ending and use that name everywhere this lesson says `image-one.jpg`. Windows treats `image-one.JPG` and `image-one.jpg` as the same file, but a web server may not, so the picture can work on your computer and be missing online. Use lowercase names and match them exactly.

> **Checkpoint.** You have `my-site` and `practice-01` side by side, there is an image file in `practice-01`, and `gh auth status` says you are logged in.

Core concepts

1. What happens when you visit a web page?

The web is a conversation between two computers. Your browser is the **client**. The computer that stores the website is the **server**.

When you type an address or click a link, the client sends a **request**: "please send me this file." The server sends back a **response** with the file's contents. For a web page, that is HTML text. The browser reads the HTML, notices it also needs a stylesheet or an image, and requests those too. Then it draws the page. While you are learning, you will often open a file straight from your own disk. Then there is no server at all; the browser reads the file directly, and the address starts with `file:///`.



Two messages: the request goes out, the response comes back. Everything else in this course happens inside one of these two boxes.

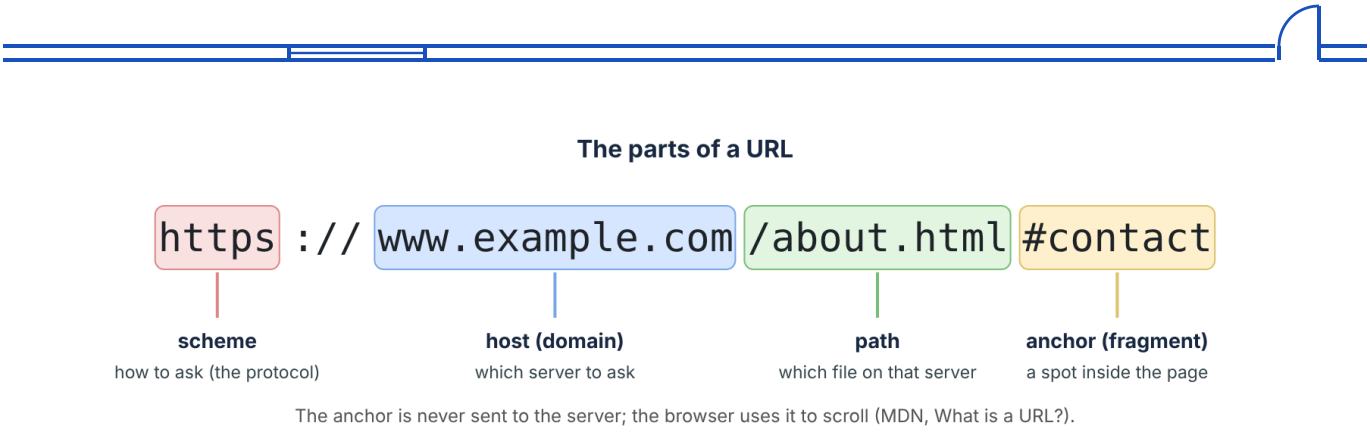
© 2026 559 Solutions LLC

Open any website and click one link. Write down which part was the request and which part was the response.

2. What are the parts of a URL?

MDN defines a URL as "the address of a unique resource on the internet" (MDN, *What is a URL?*).

Read a URL left to right. The **scheme** (`https`) tells the browser which protocol to use to ask. After `://` comes the **host**, also called the domain (`www.example.com`), which says which server to ask. Then the **path** (`/about.html`) says which file on that server. An optional **anchor** (`#contact`) points to a spot inside the page. MDN notes that the part after `#` "is never sent to the server with the request"; the browser uses it to scroll.



The anchor is never sent to the server; the browser uses it to scroll (MDN, What is a URL?).

One URL, four parts. You will write paths and anchors yourself in this module.

Look at your browser's address bar right now. Point to the scheme, the host and the path.

3. How do you create, save and see a file in VS Code?

An HTML page is a text file whose name ends in `.html`. The home page of a site is called `index.html` by convention.

In VS Code, use **File → Open Folder…** and choose the folder you want to work in. The Explorer panel on the left lists its files. Use the **New File** button at the top of the Explorer, type the full name including `.html`, and press Enter. When a tab shows a dot instead of an ×, the file has unsaved changes. Press **Ctrl+S** to save. To see the page, open `index.html` in Edge or Chrome (double-click it in File Explorer, or drag it onto a browser window). After each save, press **F5** to reload. From then on you work in a loop: **edit → save → refresh**. If the browser does not show your change, the most common reason is that you did not save. The next thing to check is that the browser shows the file you are editing: its address bar should end with that folder and file name, for example `practice-01/index.html`.

© 2026 559 Solutions LLC

The first page in the next concept was opened from disk, which is why its address starts with `file:///`.

You will create your first file in the Exercise. Remember the loop: edit, **Ctrl+S**, then F5 in the browser.

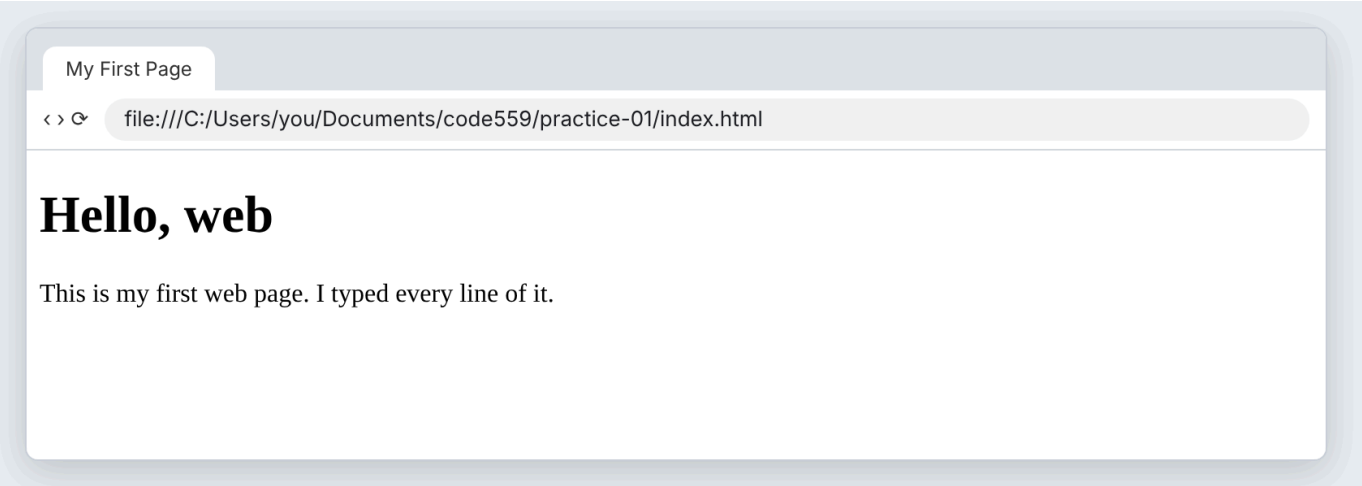
4. What does every HTML document need?

HTML is made of **elements**. Most elements have an opening tag, content and a closing tag: `<p>Hello</p>`. The closing tag has a `/`.

Every page starts with the same skeleton. `<!DOCTYPE html>` tells the browser this is modern HTML. `<html lang="en">` wraps everything and states the page's language, which MDN says helps search engines and people using screen readers. The `<head>` holds information *about* the page: `<meta charset="utf-8">` sets the character encoding so any language's characters display correctly; `<meta name="viewport" content="width=device-width, initial-scale=1">` tells phones to use their real screen width instead of pretending to be a wide desktop; `<title>` is the text in the browser tab. The `<body>` holds everything visitors see.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>My First Page</title>
  </head>
  <body>
    <h1>Hello, web</h1>
    <p>This is my first web page. I typed every line of it.</p>
  </body>
</html>
```


© 2026 559 Solutions LLC



The `<title>` appears on the tab, not on the page. The `<h1>` and `<p>` are the only things in the body. Measured at 800 px wide. (page real: rendered in Chromium; browser frame drawn)

Predict: if you change only the text inside `<title>`, what changes on screen? (Only the tab.)

5. How do headings, paragraphs and lists work?

Headings run from `<h1>` (most important) to `<h6>`. Paragraphs are `<p>`. A bulleted list is an **unordered list**, `<ul>`, with one `<li>` (list item) per bullet.

Use one `<h1>` per page for its main title, then `<h2>` for sections and `<h3>` inside those. Choose a heading level for its meaning, not its size; size is CSS's job in Module 2. Every `<li>` must sit inside the `<ul>`.

```
<h2>Three favorites</h2>
<ul>
  <li>A place I like to walk</li>
  <li>A song I play on repeat</li>
  <li>A food I could eat every week</li>
</ul>
```

You will see this exact list rendered in the Exercise.

Without looking, write the opening and closing tags of a list with two items on paper.

6. How does a link work?

A link is an `<a>` (anchor) element. Its `href` attribute holds the address to go to. The text between the tags is what people click.

© 2026 559 Solutions LLC

`href` can be a full URL (`https://developer.mozilla.org/`), a **relative path** to another file in your site (`about.html`), or an anchor on the same page (`#contact`). For a same-page link, give the target element an `id` and put `#` plus that id in the `href`. MDN puts it this way: "Elements with an `id` attribute in the document automatically create a document fragment that can be linked to" (MDN, *Creating links*). An `id` must be unique on its page.

```
<a href="#contact">Contact</a>
...
<section id="contact">
  <h2>Contact</h2>
</section>
```

The Exercise page uses a full URL, and the pages in Concept 8 use relative paths and an anchor.

Predict: what happens if the `href` says `#contacts` but the `id` is `contact`? (The link does nothing, because no element has that id.)

7. How does an image work, and what is `alt` for?

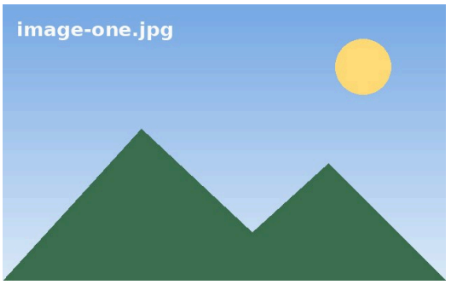
An image is an `<img>` element. It has no closing tag. `src` says which file to show. `alt` describes the picture in words.

`src` is a path, just like `href`; if the image sits in the same folder as the page, the path is just the file name. `alt` is read aloud by screen readers and is shown when the image cannot load. Write what the picture shows, as you would describe it to someone on the phone.

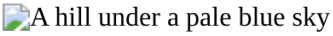
```
<img src="image-one.jpg" alt="A hill under a pale blue sky" width="320">
```


© 2026 559 Solutions LLC

src is right



src is misspelled



One wrong letter in `src` and the picture is gone. The `alt` text is what remains, so make it useful. (real: rendered in Chromium)

Write `alt` text for the picture you copied into `practice-01`. Keep it to one short sentence.

8. What are semantic elements, and how do pages share navigation?

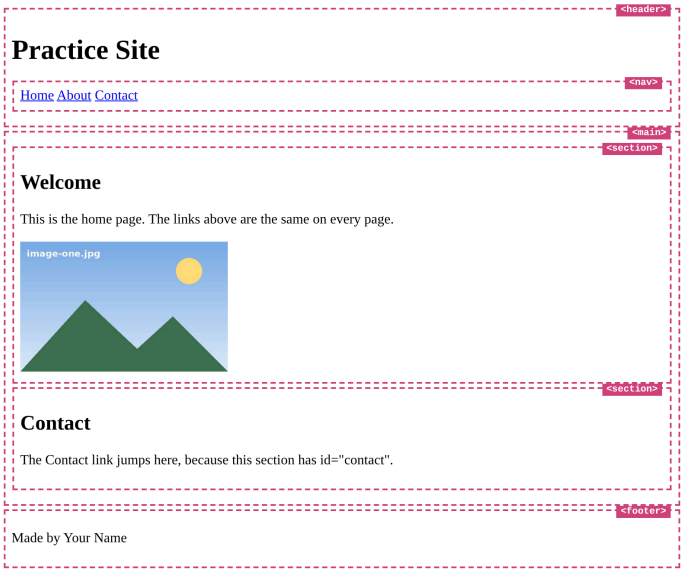
Semantic means "says what it is." `<header>`, `<nav>`, `<main>`, `<section>` and `<footer>` look like nothing at all on their own, but they tell browsers, search engines and screen readers what each part of the page is for.

A typical page has a `<header>` with the site name and a `<nav>` of links, one `<main>` with the page's own content split into `<section>`s, and a `<footer>`. A site with several pages repeats the same header, nav and footer on every page. In plain HTML you do that by **copying the same block into each file**. Links between your pages use relative paths: `about.html` means "the file called about.html in the same folder as this page." Because each page has its own copy of the navigation, you must **click every link on every page** after any change. A link to `index.html#contact` goes to another page *and* jumps to a spot on it. That is why the shared nav below uses `index.html#contact`, not just `#contact`: the Contact section exists only on the Home page. On the About page, a plain `#contact` looks for `id="contact"` on the About page, finds nothing, and the page does not move. `index.html#contact` works from both pages.

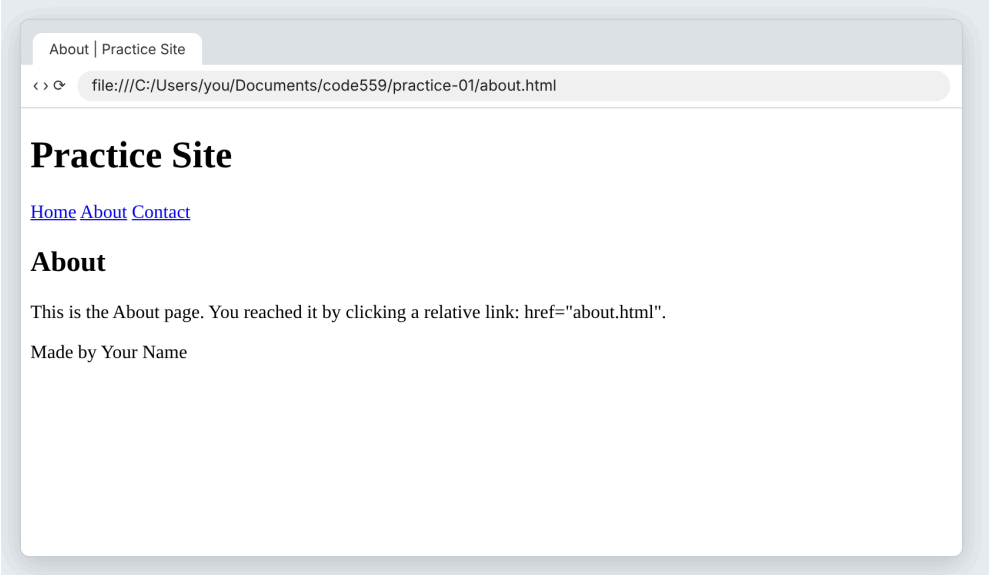
```
<header>
  <h1>Practice Site</h1>
  <nav>
    <a href="index.html">Home</a>
    <a href="about.html">About</a>
    <a href="index.html#contact">Contact</a>
```


© 2026 559 Solutions LLC

```
  </nav>
</header>
```



The same page with each semantic element outlined and labeled. The outlines were added for this picture only; without CSS these elements are invisible. Measured at 800 px wide. (real: rendered in Chromium)



*Captured after Chromium clicked the **About** link on the home page. Same header and links, different main content: that is shared navigation. (page real: rendered in Chromium; browser frame drawn)*

For a two-page site, write down how many links you must click to test the navigation fully. (Three links × two pages = six clicks.)

© 2026 559 Solutions LLC

9. How do you publish your first page?

Your pages live only on your computer. To give them a public address you need three things: a Git history of your folder, a copy of that history on GitHub (a **repository**), and **GitHub Pages**, which serves a repository's HTML as a website.

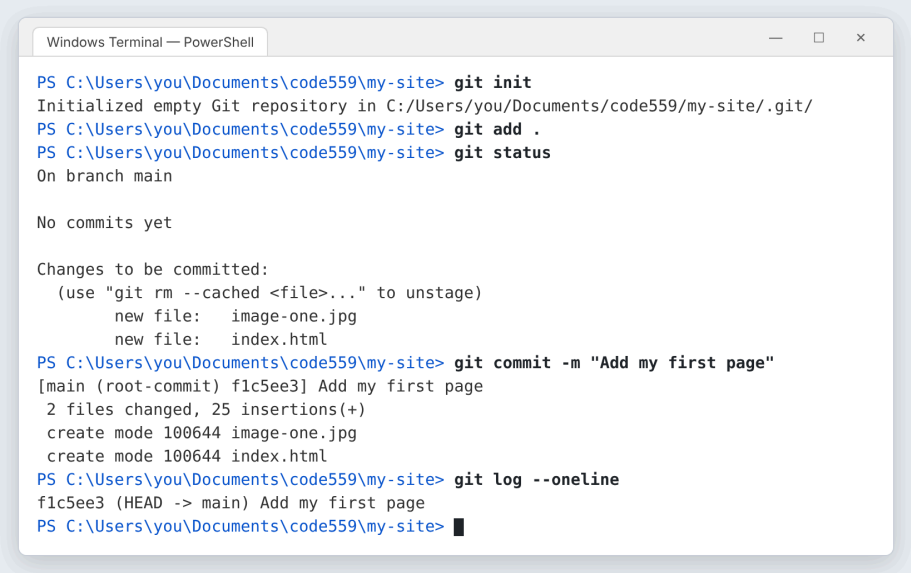
Start with the free **GitHub Skills** exercise *Introduction to GitHub*. Its page says it takes "less than one hour," needs no installs, and has the prerequisite "None." It introduces repositories, branches, commits and pull requests, and its four steps are: create a branch, commit a file, open a pull request, merge your pull request. You start it with its **Copy Exercise** button, which makes your own copy of the exercise repository; the page suggests making it public. You will not use branches or pull requests on your own site until Module 12, but seeing the whole cycle once now makes the rest of the course easier to follow.

Then you save your own folder's history with Git, on your computer:

```
git init
git add .
git commit -m "Add my first page"
```

`git init` turns the folder into a repository. `git add .` stages every file in the folder (the `.` means "this folder"). `git commit` saves a snapshot with a message. `git status` and `git log --oneline` show you what Git sees.

© 2026 559 Solutions LLC



`git add .` prints nothing when it works; `git status` proves the files are staged; `git log --oneline` shows your first commit; `(HEAD -> main)` marks the newest commit on `main`, the branch you are on (Module 12 explains it). Your commit id (the seven characters) will differ. (real output, rendered; captured on Linux, with the folder path changed to the course folder)

On Windows, `git add` may also print a warning about line endings (LF and CRLF). It is not an error, and you can carry on. Whether you see it depends on your Git settings.

Then one GitHub CLI command creates the repository on GitHub and uploads your commit:

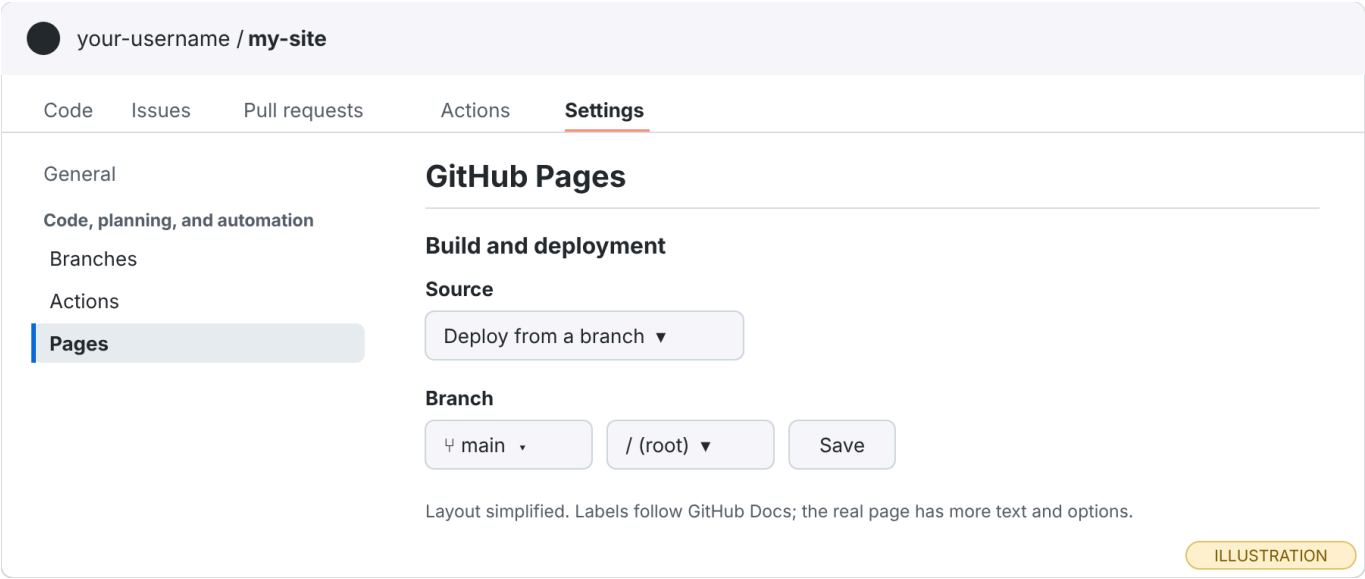
```
gh repo create my-site --public --source=. --remote=origin --push
```

The GitHub CLI manual describes the flags: `--public` "Make the new repository public"; `--source` "Specify path to local repository to use as source" (`.` is this folder); `--remote` "Specify remote name for the new repository"; `--push` "Push local commits to the new repository." This lesson cannot show you this command's output, because it needs your own signed-in account. When it works, it prints a few lines that start with ✓: one says the repository was created, one says a remote was added, and the last says your commits were pushed.

Finally, turn on Pages. GitHub Docs (*Configuring a publishing source for your GitHub Pages site*) gives these steps: on your repository, click **Settings**; in the **Code, planning, and automation** section of the sidebar, click **Pages**; under **Build and deployment**, under **Source**, select **Deploy from a branch**;

© 2026 559 Solutions LLC

choose the branch **main** and the folder **/ (root)**; click **Save**. A project site's address has the form `https://your-username.github.io/my-site/` (GitHub Docs, *What is GitHub Pages?*). Pages is free for public repositories.



Where the Pages settings live. Labels follow GitHub Docs; the real page has more on it and GitHub changes its layout from time to time.

Your Pages settings page may look slightly different from this drawing. The names **Settings**, **Pages**, **Deploy from a branch**, **main**, **/ (root)** and **Save** are what matter.

Before you run anything, write down what each of the four commands above does in five words or fewer.

Exercise

Your hands, about 75 minutes (estimate). Steps 1 to 10 work in `practice-01`. Steps 11 to 16 put a copy of your page in `my-site` and publish it. Type every line. Do not paste. Typing is how you learn where the angle brackets and slashes go.

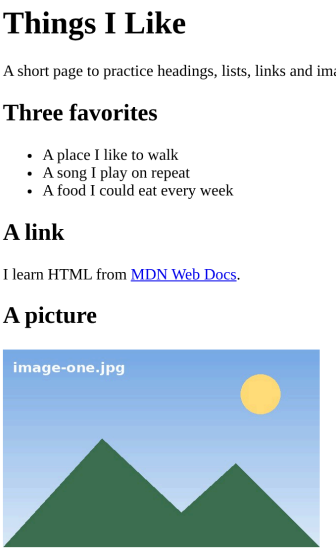
1. In VS Code, **File → Open Folder...** and choose `practice-01`. Check the Explorer shows your `image-one.jpg`.
2. Create a new file called `index.html`.
3. **Predict.** Before you type the body, sketch on paper what you expect to see: how many headings, how many bullets, one link, one picture.

© 2026 559 Solutions LLC

4. Type the skeleton from Concept 4. Change the `<title>` to `Practice Page`.

5. Inside `<body>`, type an `<h1>`, a `<p>`, and three short parts. Each part is just an `<h2>` and its content, with no `<section>` yet: one `<h2>` with a `<ul>` of three `<li>` items, one with a `<p>` containing a link to `https://developer.mozilla.org/`, and one with your `<img>`, its `alt` text and `width="320"`, as in Concept 7. Without a width, a phone photo shows at its full size, often thousands of pixels wide, and the page scrolls sideways. Choose your own words.

6. **Run.** Save with Ctrl+S. Open `index.html` in Edge or Chrome. Compare with your prediction.

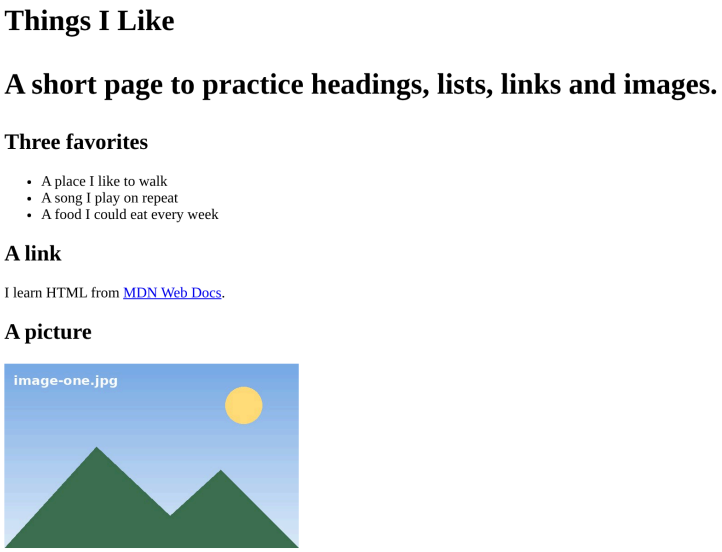


One example of a finished practice page. Yours will have your words and your picture. Measured at 800 px wide. (real: rendered in Chromium)

7. **Modify.** Add a fourth `<li>`. Save, refresh, and count the bullets. Click the link and use the browser's Back button.

8. **Break it on purpose.** Delete the closing `</h1>` tag. Save and refresh. Before you look, predict what will happen to the paragraph under it.

© 2026 559 Solutions LLC



With `</h1>` missing, the browser keeps the heading open, so the paragraph becomes part of it. The browser stops the heading when it meets the next `<h2>`, so the rest of the page looks normal. (real: rendered in Chromium)

The browser did not show an error. It guessed. HTML mistakes rarely produce a message; they produce a page that looks wrong. Put `</h1>` back, save and refresh.

9. **Break it again.** Change `src="image-one.jpg"` to `src="imge-one.jpg"`. Save and refresh. You should see your `alt` text where the picture was, as in Concept 7. Fix it.

10. **Debug.** Change the link's `href` to `https://developer.mozilla.orgg/` (an extra `g`). Save, refresh and click. Instead of MDN you get the browser's own error page, because no server answers at that host. Which part of the URL was wrong: scheme, host or path? Fix it and click again.

Publish your first page. Do the GitHub Skills exercise from Concept 9 first if you have not yet. Then:

11. **Copy your page into `my-site`.** In File Explorer, open your course folder. `my-site` is the folder you made in Module 0, next to `practice-01`; if it is missing, create it. Copy `index.html` and `image-one.jpg` from `practice-01` into `my-site`. Keep the name `index.html`: GitHub Docs says "GitHub Pages will look for an `index.html` file as the entry file for your site" (*Troubleshooting 404 errors for GitHub Pages sites*).

12. **Open `my-site` in VS Code. File → Open Folder...** and choose `my-site`. Then **Terminal → New Terminal**. Check the prompt: it must end in `my-site>`. If it ends in `practice-01>` or anything else, open `my-site` again before you type any Git command. If VS Code asks whether you trust this folder, trust it: it is your own.

© 2026 559 Solutions LLC

13. **Make your first commit.** Type these commands one at a time:

    ```
    git init
    git add .
    git commit -m "Add my first page"
    git log --oneline
    ```

    `git log --oneline` should show one commit with your message, as in the picture under Concept 9. If `git commit` says "Please tell me who you are," you skipped part of Module 0: do its step 8 (all three lines). Then type `git status`. If it says `On branch master`, type `git branch -m main`, so your branch has the name GitHub Pages looks for in step 15. Then run the `git commit` line again.

14. **Create the repository and upload.** Type:

    ```
    gh repo create my-site --public --source=. --remote=origin --push
    ```

    Look for the ✓ line that says your commits were pushed. If you get an error instead, run `gh auth status` and check that you are logged in (Module 0, step 9).

15. **Turn on GitHub Pages.** In your browser, open `https://github.com/your-username/my-site` (use your own username). Click **Settings**, then **Pages** in the sidebar. Under **Build and deployment**, under **Source**, choose **Deploy from a branch**. Under **Branch**, choose **main** and **/ (root)**, then click **Save**.

16. **Check that it worked.** GitHub Docs says "It can take up to 10 minutes for changes to your site to publish" (*Creating a GitHub Pages site*). Wait a few minutes, refresh the Pages settings page, and click **Visit site**. Or type `https://your-username.github.io/my-site/` into the address bar. You should see your practice page with its picture. Click its MDN link to test it. If you see a "404" page instead, wait a few more minutes and press F5. If the page shows but the picture is missing online, check capital letters: change the `src` in `my-site/index.html` to match the picture's file name exactly, save, and use the routine in the "Save your work" box below. (Do not fix it by renaming the file: on Windows, Git keeps remembering the old name when only capital letters change.) GitHub Docs gives the same warning for the home page: `Index.html` will not work in place of `index.html` (*Troubleshooting 404 errors for GitHub Pages sites*).

> **Save your work: the routine you use from Module 2 on.** Whenever you change a file in `my-site`, put the change online with the same five steps. Work in a terminal whose prompt ends in `my-site>`:
>
> ```
> git status
> git add .
> ```


© 2026 559 Solutions LLC

```
git commit -m "Say what you changed"
git push
```

`git status` lists the files you changed, for example `modified: index.html`. `git add .` stages them. `git commit` saves a snapshot with your message. `git push` uploads the new commit to GitHub; the `--push` in step 14 already connected your folder to GitHub, so plain `git push` is enough. The fifth step is to **check the live page**: open your site address and press F5. A change can take a few minutes to appear. Try it once now: change one word in `my-site/index.html`, save, and run the routine.

Common misconception

"If the page looks right in my browser, the HTML is correct."

What would you say? Hint: think about step 8 of the Exercise, where the browser quietly guessed, and about who reads `alt` text.

Summary

A browser (the client) requests files from a server and draws the HTML it gets back. A URL names the scheme, the host, the path and optionally an anchor inside the page. You work in a loop of edit, save and refresh, and an unsaved file is the most common reason a change does not show. Every page has the same skeleton: doctype, `html` with `lang`, a `head` with charset, viewport and title, and a `body`. Headings give a page its outline, paragraphs hold text, and `ul` with `li` makes a bulleted list. An `a` element's `href` can point to another site, another page by a relative path, or an `id` on the same page. An `img` needs a correct `src` and a useful `alt`. Semantic elements name the parts of a page, and shared navigation is copied onto every page and tested link by link. Git records your folder's history, `gh repo create` puts it on GitHub, and GitHub Pages serves it at a public address. After that, every change goes online with one routine: status, add, commit, push, then check the live page.

Check your understanding

1. In `https://www.example.com/about.html#contact`, which part is never sent to the server, and what does the browser do with it?
2. You changed your page, refreshed, and nothing changed. Name two things to check, in order.
3. **What would you test** after adding a third page to the Practice Site from Concept 8, so that its nav has four links, and how many clicks does it take?

© 2026 559 Solutions LLC

Before next module

- [ ] Your practice page has headings, a list, a link and an image, and you broke and fixed it
- [ ] You finished the GitHub Skills *Introduction to GitHub* exercise
- [ ] `git log --oneline` in `my-site` shows at least one commit
- [ ] Your `my-site` repository exists on GitHub and Pages is set to deploy from `main`, `/ (root)`
- [ ] Your first page opens at `https://your-username.github.io/my-site/`, with its picture, and its link works
- [ ] You ran the save routine once: git status → git add . → git commit -m "..." → git push → check the live page

Go further

- MDN: *What is a URL?*, *What's in the head? Web page metadata*, *Creating links*, and the reference page for `<meta name="viewport">`
- MDN: search "HTML elements reference" for `header`, `nav`, `main`, `section` and `footer`
- GitHub Skills: *Introduction to GitHub* (github.com/skills/introduction-to-github)
- GitHub Docs: *What is GitHub Pages?* and *Configuring a publishing source for your GitHub Pages site*
- GitHub CLI manual: `gh repo create`
- freeCodeCamp: the HTML section of the Responsive Web Design curriculum

What to keep

Keep your `practice-01` folder, the address of your live page, and a note of the four commands you ran to publish it. Then answer for yourself: **which step would be hardest to repeat without this packet, and why?**

Code559 is a 559 Solutions project. Portions adapted from Visual Studio Code documentation (CC BY 3.0 US), GitHub Docs (CC BY 4.0), and GitHub Skills (MIT). Screenshots used with permission from Microsoft.

Sources checked 2026-10-09: MDN "What is a URL?"; MDN "What's in the head? Web page metadata"; MDN "`<meta name="viewport">`"; MDN "Creating links"; GitHub Skills "Introduction to GitHub" (github.com/skills/introduction-to-github); GitHub Docs "What is GitHub Pages?"; GitHub Docs "Configuring a publishing source for your GitHub Pages site"; GitHub CLI manual "gh repo create". Checked 2026-10-11: GitHub Docs "Creating a GitHub Pages site"; GitHub Docs "Troubleshooting 404 errors for GitHub Pages sites"; GitHub CLI source code (cli/cli on GitHub: `pkg/cmd/repo/create/create.go` and `git/client.go`, for what `gh repo create --push` prints and does); Apple Support "Manage location metadata in Photos" (Personal Safety User Guide).

© 2026 559 Solutions LLC

PART 2

Project: Home and About pages

Your hands, about 75 minutes (estimate). In the Exercise you published your practice page as the first page of `my-site`. Now you replace it with the start of the personal website you will build all course: a Home page and an About page. You use only the HTML from this module. The pages will look plain, in black and white with blue links. That is correct. Styling starts in Module 2.

The brief

When you finish, `my-site` holds two pages and one picture: `index.html` (Home), `about.html` (About) and `image-one.jpg`. Both pages are online at `https://your-username.github.io/my-site/`, and a visitor can click between them.

Each page has the full skeleton from Concept 4: `<!DOCTYPE html>`, `<html lang="en">`, and a `<head>` with `<meta charset="utf-8">`, the viewport line and a `<title>`. Each page has its own title, so the two browser tabs are easy to tell apart. Use `Home - Your Name` and `About - Your Name`, with your own name or the name you want on your site.

Each page's body has four parts, in this order:

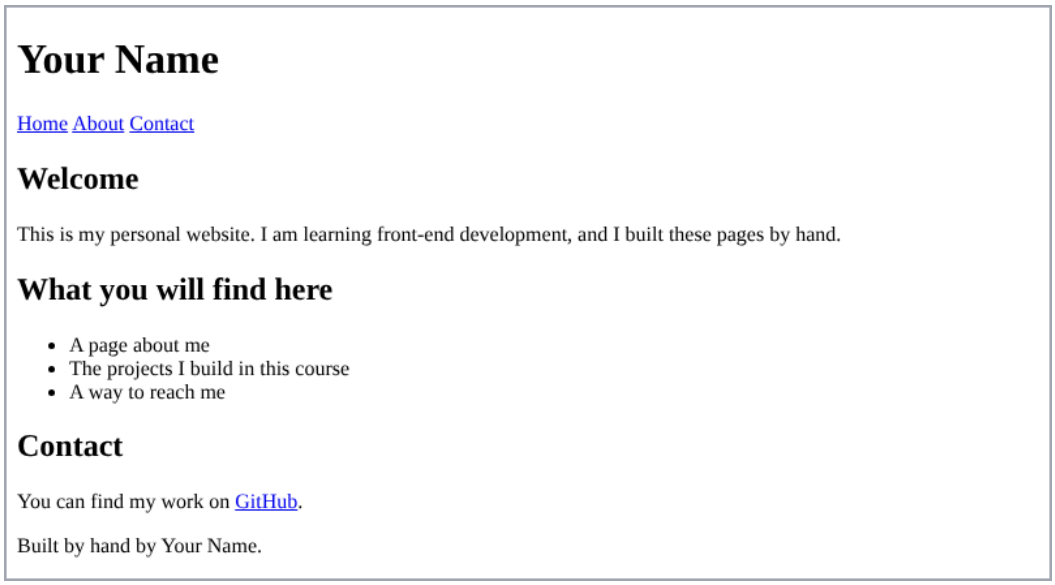
- A `<header>` with an `<h1>` that holds your site name, and a `<nav>` with three links. The link words are exactly **Home**, **About** and **Contact**. The header is the same, letter for letter, on both pages.
- One `<main>` with the page's own content in `<section>` elements. Each section starts with an `<h2>`.
- A `<footer>` with one short line, for example `Built by hand by Your Name.` The footer is also the same on both pages.

The Home page's main area has three sections: **Welcome** (a short paragraph about the site), **What you will find here** (a list of three things), and **Contact** (a paragraph with one link, for example to your GitHub profile at `https://github.com/your-username`). The Contact section has `id="contact"`, so the Contact link can jump to it.

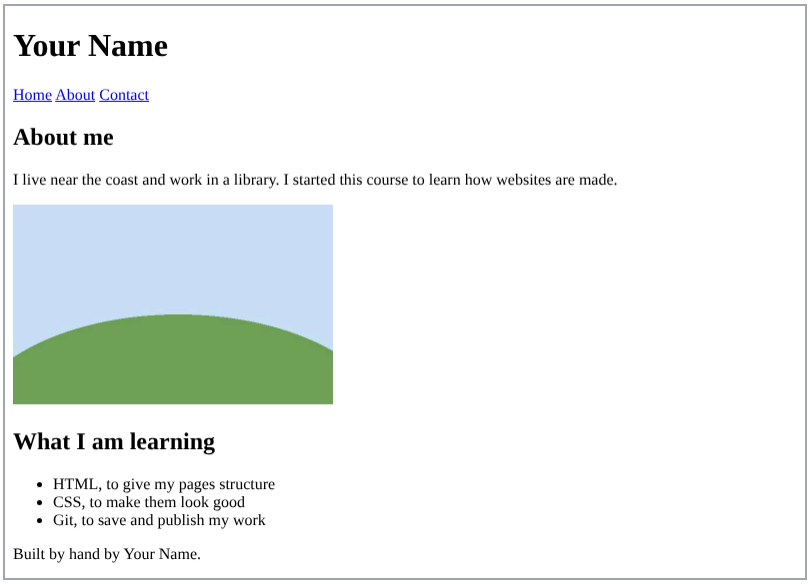
The About page's main area has two sections: **About me** (a paragraph about you, and your picture with `alt` text and `width="320"`) and **What I am learning** (a list of three things).

A visitor can do four things: open your address and see the Home page; click **About** to get the About page; click **Home** to come back; and click **Contact** from either page to reach the Contact section of the Home page. All the words are yours. Only the headings, the link words and the file names above need to match.

© 2026 559 Solutions LLC



One example of a finished Home page. With no CSS, the three nav links sit on one line, and the header, main and footer are invisible: only their content shows. Measured at 800 px wide. (real: rendered in Chromium at 800 px)



The About page. The header and footer match the Home page exactly; only the main content changed. (real: rendered in Chromium at 800 px)

The steps

Type every line. Do not paste. There is one exception, and it is on purpose: in step 7 you copy the header and footer from one page to the other, because that is how plain HTML pages share navigation (Concept 8).

© 2026 559 Solutions LLC

1. **Open `my-site`.** In VS Code, **File → Open Folder...** and choose `my-site` in your course folder. The Explorer should show `index.html` and `image-one.jpg` from the Exercise. Open a terminal with **Terminal → New Terminal** and check that the prompt ends in `my-site>`. You will need it at the end.

2. **Plan on paper.** Write your site name, the three things for the Home page list, and the three things for the About page list. Then sketch both pages as boxes: header, main with its sections, footer.

3. **Predict.** Next to your sketch, write how many headings each page will show, how many bullets, and how many links. You will check this in step 6.

4. **Start the new Home page.** Open `index.html`. Your practice page is in it. You are replacing it, so select everything (**Ctrl+A**), delete it, and type the skeleton below. Your practice page is not lost: it is still in `practice-01`, and Git kept the published copy in your first commit.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Home - Your Name</title>
  </head>
  <body>
    <header>
      <h1>Your Name</h1>
      <nav>
        <a href="index.html">Home</a>
        <a href="about.html">About</a>
        <a href="index.html#contact">Contact</a>
      </nav>
    </header>
    <main>
      <section>
        <h2>Welcome</h2>
        <p>One or two sentences about your site.</p>
      </section>
      <!-- Continue alone: a section "What you will find here" with a list of
three items,
           then a section with id="contact", an h2 "Contact", and a paragraph
with one link. -->
    </main>
    <footer>
      <p>Built by hand by Your Name.</p>
    </footer>
  </body>
</html>
```


© 2026 559 Solutions LLC

Put your own name in the `<title>`, the `<h1>` and the footer. The HTML comment marks where you continue alone. When you have typed your two sections, delete the comment. A comment never shows on the page, but a leftover one is clutter.

5. **Check your tags.** Before you look at the page, read your code from top to bottom. Every `<section>` needs its `</section>`, every `<li>` sits inside the `<ul>`, and the `id` is written `id="contact"`, all lowercase, with no `#`. The `#` belongs only in the link.

6. **Run.** Save with Ctrl+S. Open `index.html` in Edge or Chrome (double-click it in File Explorer, or drag it onto a browser window). After each save, press **F5** to reload. Compare the page with your prediction from step 3. The tab should say `Home - Your Name`.

7. **Make the About page.** In the Explorer, create a new file called `about.html`. Type the skeleton (doctype, `html`, `head` with the two `meta` lines) and give it the title `About - Your Name`. Then copy the whole `<header>…</header>` block from `index.html` and paste it as the first thing inside `<body>`. Do the same with the `<footer>…</footer>` block, at the end of the body. Between them, type the `<main>`:

```
<main>
  <section>
    <h2>About me</h2>
    <p>Two or three sentences about you.</p>
    <img src="image-one.jpg" alt="Describe your picture in one short sentence"
width="320">
  </section>
  <!-- Continue alone: a section with an h2 "What I am learning" and a list of
three items. -->
</main>
```

   Write real `alt` text for your picture, as in Concept 7. If your picture is a `.png`, use its real name here, as you did in the Exercise.

8. **Run again.** Save, open `about.html` in the browser, and press F5. You should see the same header and footer as the Home page, your About content, and your picture.

9. **Modify.** Add a fourth item to the About page's list. Save and refresh. Then change only the text inside `<title>` on the About page. Save and refresh, and check that only the tab changed. Put back the title you want.

10. **Test every link.** Start on the Home page. Click each nav link, use the browser's **Back** button, and tick the box. Then do the same from the About page. Each time, look at the end of the address bar as well as the page.

    - [ ] Home page → **Home**: the Home page reloads; the address ends in `index.html`
    - [ ] Home page → **About**: the About page opens; the address ends in `about.html`

© 2026 559 Solutions LLC

- [ ] Home page → **Contact**: the address ends in `index.html#contact`
- [ ] About page → **Home**: the Home page opens
- [ ] About page → **About**: the About page reloads
- [ ] About page → **Contact**: the Home page opens; the address ends in `index.html#contact`
- [ ] Home page → the link in the Contact section opens the right website

That is six nav clicks: three links on two pages (Concept 8). If your Home page is short, the Contact link may not seem to move the page, because there is nowhere to scroll. That is fine when the address ends in `#contact`. To see the jump, make the browser window shorter and click it again.

11. **Break it on purpose.** On the About page only, change the Contact link to `href="#contact"`. Save and refresh. Predict before you click: what will happen? Then click **Contact**.

    The address now ends in `about.html#contact`, and you are still on the About page. Nothing on the screen moves, and there is no error message. The browser looked for `id="contact"` on the About page and found nothing (Concept 8). This is why you test every link on every page: the same nav can work on one page and fail on the other. Put back `href="index.html#contact"`, save, refresh, and click Contact again to check the fix.

12. **Debug.** On the Home page, change the About link to `href="abuot.html"`. Save, refresh and click it. Instead of your About page, the browser shows an error page, because there is no file with that name. Which part was wrong: the link words, or the path? Fix the path and test the link again.

13. **Compare the two headers.** Switch between the `index.html` and `about.html` tabs in VS Code and read the two `<header>` blocks line by line. They must match exactly. Do the same for the footers.

14. **Put it online.** Go to Part 3, *Finishing work in Git*, and publish both pages with the save routine. Then run the link test from step 10 again on the live site.

© 2026 559 Solutions LLC

PART 3

Finishing work in Git

Your new pages work on your computer. Now you put them online with the save routine from the lesson's "Save your work" box: `git status`, `git add .`, `git commit`, `git push`, then check the live page. You use this routine at the end of every module from now on.

Items marked **NEW** are new in this part. The others repeat what you did in the Exercise.

1. **Check the terminal.** In VS Code, with `my-site` open, look at the terminal prompt. It must end in `my-site>`. If it does not, use **File → Open Folder...**, choose `my-site`, and open a new terminal.

2. **See what changed (NEW: `git status` after a change).** Type:

   ```
   git status
   ```

   Git lists the changes it sees but has not saved yet. You should see `index.html` under **Changes not staged for commit**, marked `modified:`, because you replaced its content. You should see `about.html` under **Untracked files**, because Git has never saved it before. `image-one.jpg` is not listed: it has not changed since your first commit, so Git has nothing new to save. If `git status` lists a file you do not expect, open it and check before you go on.

3. **Stage the changes.** Type:

   ```
   git add .
   ```

   It prints nothing when it works. (A warning that mentions LF and CRLF is harmless.) Run `git status` again. Now both files are under **Changes to be committed**: `new file: about.html` and `modified: index.html`.

4. **Commit.** Type:

   ```
   git commit -m "Add Home and About pages"
   ```

   Git prints a short summary. The first line starts with `[main` and a seven-character commit id, then your message. A later line says `create mode 100644 about.html`, which means a new file was saved. Your commit id and numbers will differ. Write messages that say what you changed, so you can find a change again later.

5. **Push (NEW: `git push` on its own).** Type:

   ```
   git push
   ```


© 2026 559 Solutions LLC

In the Exercise, `gh repo create … --push` uploaded your first commit and connected your folder to GitHub. Now plain `git push` is enough. It prints a few lines; the last one ends in `main -> main`, which means your `main` branch on GitHub now has your new commit.

6. **Check that Git is done.** Type `git status` once more. It should say `nothing to commit, working tree clean`. Then type `git log --oneline`. You should see two commits: `Add Home and About pages` on top, and `Add my first page` under it.

7. **Check the live page (NEW: testing the live site).** GitHub Docs says "It can take up to 10 minutes for changes to your site to publish" (*Creating a GitHub Pages site*). Wait a few minutes, then open `https://your-username.github.io/my-site/` and press F5. You should see your new Home page, not the practice page. If you still see the practice page, wait a few more minutes and press F5 again.

8. **Test the live links.** Run the link test from Project step 10 again, on the live site. Click all three nav links on both pages, and check that the picture shows on the About page. A link or picture can work on your computer and fail online, usually because of capital letters in a file name (see *Troubleshooting*).

> **Checkpoint.** `git status` says `nothing to commit, working tree clean`. `git log --oneline` shows two commits. Your live address shows your Home page, and all six nav clicks work there too.

© 2026 559 Solutions LLC

PART 4

Troubleshooting: five common problems

Each problem has the symptom you see, why it happens, and the fix. Try the fix, then run the link test from Project step 10 again.

1. The About link works on your computer but gives a 404 online

Symptom. On your computer, clicking **About** opens your About page. On your live site, the same click shows a GitHub "404" page.

Why. The name in the link and the name of the file differ only in capital letters, for example `href="about.html"` and a file called `About.html`. Windows treats them as the same file, so the link works on your computer. A web server may not, so online the file is not found. GitHub Docs gives the same warning for the home page: `Index.html` will not work in place of `index.html` (*Troubleshooting 404 errors for GitHub Pages sites*).

Fix. Look at the file name in the VS Code Explorer and make every link match it exactly, on both pages. Save, run the save routine, and test again online. As in the lesson's Exercise step 16, fix the link, not the file name: on Windows, Git keeps remembering the old name when only capital letters change. From now on, name every file in lowercase.

2. The live site still shows the old practice page

Symptom. You pushed, opened your address, and still see the practice page from the Exercise.

Why. Usually GitHub Pages has not finished publishing yet. GitHub Docs says "It can take up to 10 minutes for changes to your site to publish" (*Creating a GitHub Pages site*). Less often, the push did not happen.

Fix. Wait a few minutes and press F5 on the live page. If it still shows the old page, type `git status` in the `my-site` terminal. If it lists changed files, you skipped `git add .` or `git commit`: finish the routine. Then open `https://github.com/your-username/my-site` in the browser. If `about.html` is not in the file list there, the push did not reach GitHub: read the message from `git push` again, and see problem 3.

3. `git push` stops with an error

Symptom. `git push` prints a line starting with `fatal:` instead of finishing with `main -> main`.

Why. There are two common causes. If the message says `No configured push destination`, this folder was never connected to GitHub: step 14 of the Exercise was run in another folder, or not at all. If the message is about signing in or permission, the GitHub CLI is not logged in.

© 2026 559 Solutions LLC

Fix. For a sign-in problem, run `gh auth status` and check that you are logged in (Module 0, step 9). For `No configured push destination`, first open `https://github.com/your-username/my-site`. If no such repository exists, run step 14 of the Exercise again in `my-site`. If the repository does exist, connect the folder to it with these two lines, using your own username:

```
git remote add origin https://github.com/your-username/my-site.git
git push --set-upstream origin main
```

After that, plain `git push` works.

4. `about.html` opens as plain text

Symptom. You double-click your new About page and see the code, or Windows calls it a "Text Document".

Why. With file extensions hidden, the file was saved as `about.html.txt`. A link to `about.html` will not find it either.

Fix. This is the same problem as in the Module 0 packet. Turn on **View → Show → File name extensions** in File Explorer, rename the file so it ends in `.html` only, and accept the warning about changing the extension. In VS Code, check the tab name after saving.

5. The picture on the About page does not show

Symptom. Where your picture should be, you see a small broken-image icon and your `alt` text.

Why. The `src` does not match the picture's file name and place. Either the name is spelled differently (`image-one.JPG`, `imageone.jpg`, `.png` instead of `.jpg`), or the picture is not in `my-site` next to `about.html`.

Fix. Check that the picture is in the VS Code Explorer under `my-site`, at the same level as `about.html`. If it is missing, copy it from `practice-01`. Then copy its exact name into the `src`, letter for letter. Save and refresh. If it works on your computer but not online, it is the capital-letter problem from problem 1.

© 2026 559 Solutions LLC

PART 5

Self-assessment

Rate yourself on each line: 1 Beginning, 2 Developing, 3 Proficient, 4 Ready to advance. Aim for at least **3 (Proficient)** on every line before you start Module 2. If a line is a 1 or a 2, reread that Core Concept and redo its Exercise or Project step.

Skill	1 Beginning	2 Developing	3 Proficient	4 Ready to advance
I can explain what the client and the server do when I open a page	☐	☐	☐	☐
I can name the scheme, host, path and anchor in a URL	☐	☐	☐	☐
I can type the full HTML skeleton from memory and say what each line is for	☐	☐	☐	☐
I can use headings, paragraphs and lists for their meaning, not their size	☐	☐	☐	☐
I can write links to another site, to another page, and to an id on a page	☐	☐	☐	☐
I can add an image with a correct src and useful alt text	☐	☐	☐	☐
I can build a page from header, nav, main, section and footer, and share the nav across pages	☐	☐	☐	☐
I can test every link on every page, and publish a change with the save routine	☐	☐	☐	☐

Three questions to answer in writing

1. In Project step 11, the Contact link worked on one page and failed on the other. Explain why, in your own words.
2. Pick one mistake you made in this module. What did the page or the terminal show, and how did you find the cause?
3. Your site gets a third page. List everything you must change and test, in order.

© 2026 559 Solutions LLC

PART 6

Answers

Try each question on your own first, in writing. Then compare. The section in brackets is where the lesson or the project explains it.

Check your understanding

1. **The part never sent to the server.** The anchor, `#contact`. MDN says the part after `#` "is never sent to the server with the request." The browser keeps it and uses it to scroll to the element with `id="contact"` once the page arrives. (Core concept 2)
2. **A change does not show.** First, check that you saved: a dot on the VS Code tab instead of an ✕ means unsaved changes. Press Ctrl+S, then F5. Second, check that the browser shows the file you are editing: the address bar should end with the right folder and file name, for example `my-site/about.html`. (Core concept 3)
3. **What would you test with a third page.** Every nav link on every page. Each page now has four links, and there are three pages, so that is four × three = twelve clicks. Before you test, check that the new link was added to the nav on all three pages, not only on the new one, and that the Contact link still says `index.html#contact` on every page. (Core concept 8)

Common misconception

"If the page looks right in my browser, the HTML is correct." It is not that simple. The browser fixes many mistakes quietly and never shows an error. In Exercise step 8, a missing `</h1>` did not stop the page; the browser guessed where the heading ended. A guess can look right in one place and wrong in another. And some readers do not see the page at all: a screen reader reads the headings, the `lang`, and the `alt` text, and a search engine reads the structure. Your page can look fine and still have a missing `alt`, a heading chosen for its size, or a broken link you have not clicked yet. Looking right is a first check, not proof. Read your tags, and test every link. (Exercise step 8; Core concepts 7 and 8)

Exercise checks

- **Step 6.** Your page shows one large heading, a paragraph, three smaller headings, a bulleted list, a link and your picture. The tab says `Practice Page`.
- **Step 7.** Four bullets after you add the fourth `<li>`. The link opens MDN, and **Back** returns to your page.
- **Step 8.** With `</h1>` missing, the paragraph under the main heading becomes as large and bold as the heading. The rest of the page looks normal, because the browser ends the heading at the next `<h2>`.
- **Step 9.** With `src="imge-one.jpg"`, the picture disappears and your `alt` text shows in its place.
- **Step 10.** The wrong part is the **host**: `developer.mozilla.orgg` is not a server that answers.

© 2026 559 Solutions LLC

- **Steps 13 to 16.** `git log --oneline` shows one commit, `gh repo create` ends with a ✓ line saying your commits were pushed, and your page opens at `https://your-username.github.io/my-site/` with its picture.

Self-assessment questions

1. **Why Contact worked on one page and failed on the other.** `href="#contact"` has no file name, so the browser looks for `id="contact"` on the page you are already on. The Home page has that id on its Contact section, so the link works there. The About page has no element with that id. So the address ends in `about.html#contact`, you stay on the About page, and nothing moves. There is no error message. `href="index.html#contact"` names the file first, so it works from every page. (The brief; Project steps 10 and 11)
2. **A mistake you made.** There is no single answer. A good answer names the mistake, quotes what the page or the terminal really showed, and says how you found the cause, for example by reading your tags top to bottom, or by checking the end of the address bar.
3. **A third page.** First, create the new file with the full skeleton and its own `<title>`. Second, copy the header and footer from `index.html` into it, and write its `<main>` with sections that each start with an `<h2>`. Third, add a link to the new page in the nav of every page, the two old ones too, so the headers still match letter for letter. Fourth, save, open the new page and press F5. Fifth, compare the headers and footers of all three pages line by line. Sixth, test every nav link on every page: four links on three pages is twelve clicks, and Contact must still end in `index.html#contact`. Last, publish and run the link test again on the live site. (The brief; Project steps 7, 8, 10, 13 and 14)

Project checks

- **Brief.** Two pages, `index.html` and `about.html`, each with the full skeleton, its own `<title>`, the same header and footer, and a `<main>` made of sections that each start with an `<h2>`. The About page has your picture with `alt` text. (The brief)
- **Step 6.** The Home page shows your site name as the largest heading, the three links on one line, your sections, and the footer line. Without CSS, the header, nav, main, section and footer elements add no boxes or colors of their own. (Step 6; Core concept 8)
- **Step 10.** All six nav clicks go where the checklist says. Contact goes to `index.html#contact` from both pages.
- **Step 11.** With `href="#contact"` on the About page, the address ends in `about.html#contact` and you stay on the About page, because the About page has no element with `id="contact"`. (Step 11; Core concept 8)
- **Step 12.** With `href="abuot.html"`, the browser shows an error page. The path was wrong; the link words were fine.
- **Finishing work.** `git status` says `nothing to commit, working tree clean`, `git log --oneline` shows two commits, and the live address shows your new Home page with working links. (Part 3, steps 6 to 8)

© 2026 559 Solutions LLC

NEXT

Module 2: CSS Foundations.
Free lesson: 559solutions.com/code559/module-02-css-foundations/

© 2026 559 Solutions LLC

About 559 Solutions

559 isn't just a name. It's where I started: 559 East 37th Street, in the Ida B. Wells Homes on Chicago's South Side. 559 Solutions carries that start into the work: plain language, steady delivery, and solutions people actually use.

The blueprint and the open door

Every 559 Solutions page is drawn like a floor plan: a wall, a window, another wall, and an open door. The plan is how we solve problems, large or small, in any corner of information technology. That covers a nonprofit's tangled spreadsheet and a company's enterprise architecture alike. Every solution moves through the same four steps:

- **Inform:** name the problem, and why it matters.
- **Explain:** show how the solution works, one idea at a time.
- **Illustrate:** draw it, so everyone sees the same plan.
- **Apply:** build it, and put it to work.

In every module, the same four steps sit behind the badges; there the first one is called Introduce. The open door is the point of the plan: a way in, for clients and learners alike.

Code559

Code559 is the teaching side of 559 Solutions. It teaches more than code. It teaches a developer's mindset: predict before you run, read the error, change one thing at a time, test it, save it, and check it live. You work with the real tools developers use every day: VS Code, Git, GitHub, the browser's DevTools and Node. You build working code you can publish, the way a developer does on the job. Think of it as a day in the life of a developer, built into every module.

No jargon. No drama. Just solutions that serve the mission.

— J. R. Wells

559solutions.com

© 2026 559 Solutions LLC